

《人工智能通识》

第八讲: Context Engineering —— AI 如何记住重要信息

周天航

重质油全国重点实验室
中国石油大学 (北京) 未来能源学院

2026 年 9 月



先回顾: 第七讲的 AI Agent, 你还能用大白话说清吗?

- Q1** Agent 是什么? 会自己规划、调工具、看结果、再决定下一步的模型
- Q2** 它靠什么动手? Function Call (第五讲) + MCP (第六讲)
- Q3** 它怎么一步步做? 感知 → 规划 → 行动 → 观察, 循环直到任务完成

上一讲结尾留了一个现象: Agent 跑到第二三十步, 上下文很快被**塞满**, 最早的任务目标、你交代的关键约束被**挤出窗口**, 它开始“跑偏”。

今天就讲这件事: 上下文里到底装了什么, 怎样让 AI **记得该记的、忘掉该忘的**。

情境：让 Agent 排查“三号加氢反应器今晚温升为什么偏快”



第 1 步, 值班工程师交代: “只看今晚 20:00 以后的数据, **别动任何设定值**, 结论要给出依据。”

第 2 ~ 25 步, Agent 调 DCS 工具拉数据、查规程、翻历史工况……每次工具返回几千 token, 全部留在对话里。

第 26 步, 它建议: “把入口温度设定值下调 3 度。”

Q: 它是“不听话”, 还是根本**看不见**你那句话了?

课堂互动① · 举手投票

你用 AI 时被“忘事”坑过吗? A. 聊长了, 它忘了开头的设定 B. 改文档改到后面, 把前面改好的又改回去 C. 没遇到过。

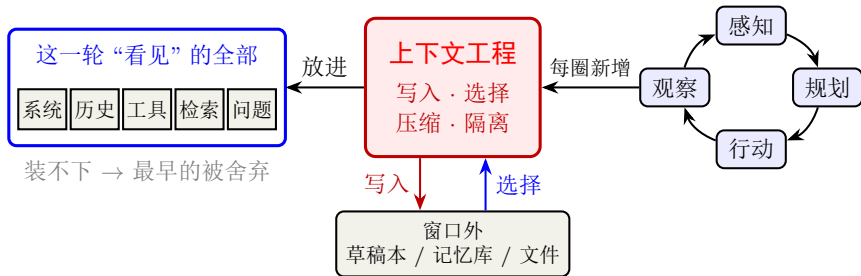
举手选一项, 1 分钟; 选 A、B 的同学各请一位用一句话说说当时发生了什么。

本讲在哪里：“窗口”与“循环”之间，缺一道闸门

第二讲：有限的窗口

本讲：中间这道闸门

第七讲：Agent 循环



第二讲讲过窗口有多大、超出会滑动忘掉开头；第七讲讲过 Agent 一圈圈循环往窗口里塞东西。

上下文工程就是这道闸门：决定什么进窗口、什么写到外面、什么压小、什么隔开。

本讲学习目标

知识目标

说清一次推理的上下文由哪些部分组成; 区分短期草稿与长期记忆; 解释写入、选择、压缩、隔离各解决什么问题; 知道“上下文腐烂”和“中间丢失”是什么现象。

能力目标

能估算多步 Agent 任务的上下文增长速度; 能把一段自由记录改写成结构化模板; 能在“全塞进去 / 摘要 / 检索”之间做出成本与效果的权衡。

素养目标

养成“关键数字回原始数据核对”的习惯; 对写进 AI 长期记忆的内容保持边界意识。

学习建议: 每讲到一个方法, 先想想值班交接、实验记录本里有没有你早就在做的同一件事。

第一部分·上下文：模型这一次推理能看到的全部信息

先把第二讲的两句话搬回来：

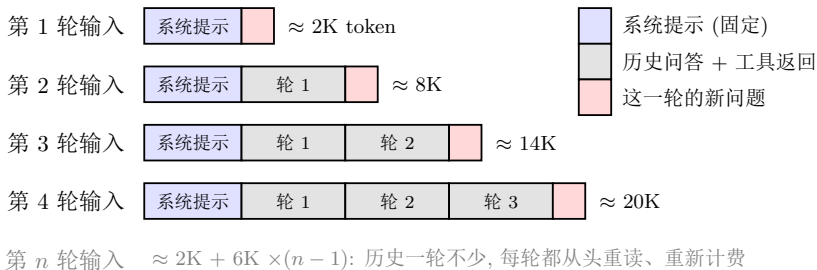
上下文长度 = 模型一次推理中能“看到”的连续 token 数；
超出窗口，就把最前面的部分舍弃 (滑动窗口)，所以聊久了会
“忘记”开头。

每次你提问，模型都重新读一遍能看到的内容，再据此生成回答。
它没有人类那样的长期记忆，也不会主动挑选、压缩、提炼——只会机械地重读输入。

所以 AI 的健忘，不是记性差，而是记忆方式太笨。

Q:那我们该怎样让它——记得该记的、忘掉该忘的？这就是上下文工程要解决的事。

每一轮, 模型都把全部上下文重新读一遍——于是它越滚越大



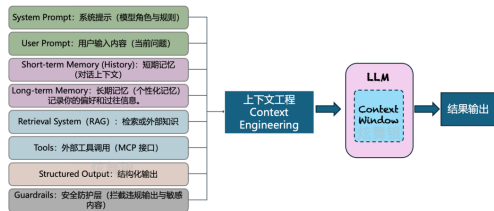
第 n 轮的输入 = 系统提示 + 前 $n - 1$ 轮的全部问答 + 这一轮的新内容。

这就是为什么: 多轮对话越聊越贵 (每轮都重新计费全部输入), 也越聊越容易挤出开头。

上下文 ≠ 你在对话框里看到的那几句话

模型在一次推理中能接触到的**全部动态信息**:

- ▶ **系统提示**: 角色、回答风格、安全与合规要求
- ▶ **用户输入**: 你这一轮说的话
- ▶ **历史对话**: 短期记忆
- ▶ **长期记忆**: 跨会话的偏好与经验
- ▶ **检索结果**: RAG 找回的文档片段
- ▶ **工具结果**: MCP / Function Call 返回的数据
- ▶ **输出格式要求、安全护栏**



配图: 一次推理的上下文由这些部分拼成,
一起送进窗口

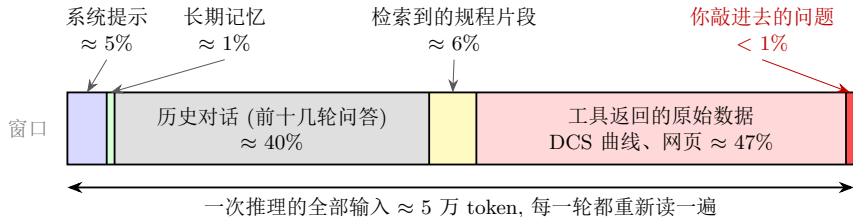
一个例子：一次“北京旅行”对话，四轮里模型各看到了什么

轮次	用户说	这一轮模型实际看到的
1	我想去北京玩，最近三天天气如何？	本轮输入 + 系统提示 + MCP 天气工具返回的实时数据
2	周末有什么好玩的活动？	本轮输入 + 第 1 轮问答 + 系统提示 + 多轮联网检索到的网页信息
3	我喜欢安静的，最好有书店或咖啡馆。	本轮输入 + 前两轮 + 长期记忆 / RAG 提取的偏好 + 系统提示
4	帮我画一张，阳光很好、有秋天感觉的。	本轮输入 + 前三轮 + 偏好记录 + 天气信息 + 系统提示 + 图像生成调用的参数

你敲进去的每一轮只有一句话，但模型每轮看到的是一**整包**：越到后面，包越大。

“长期记忆”和“工具返回”在对话框里是看不见的，但它们确实占着窗口。

把窗口切开看：谁占了多少？



一个跑了十几步的 Agent, 你自己敲的字往往不到 1%;
真正的大头是工具返回的原始数据和一轮轮累积的历史。

比例为示意, 随任务类型差异很大; 但“用户输入很小、工具与历史很大”是普遍规律。

化工同学的版本：值班助手回答一次提问，窗口里都有什么

组成部分	在“值班助手”里对应什么	大约 token
系统提示	角色 (工艺值班助手); 规则 (不得建议修改设定值; 结论必须带位号与时间)	2 000
用户输入	“三号反应器今晚温升为什么偏快?”	30
工具结果	DCS 返回该反应器 12 个位号最近 2 小时的数据	20 000
检索结果	《加氢装置操作规程》第 5 章“反应器温升异常”3 段	3 000
历史对话	前 10 轮排查的问答与工具返回	25 000
长期记忆	这位工程师习惯看 °C 而非 K; 要求结论附依据	300

加起来约 5 万 token ——而工程师那句 30 个 token 的问题，只是冰山露出水面的一角。

Q:如果再排查 20 步，会发生什么？下一页你来算。

数一数 (纸笔估算, 2 分钟)

课堂互动②

接上页的值班助手: 系统提示 2K, 规程片段 3K 固定不变; 每排查一步, 工具返回约 4K token, 并且**全部保留**在历史里。

(1) 第 N 步时, 上下文大约多少 token? 写出式子。 (2) 128K 的窗口大约在第几步被装满?

算完报数, 看全班的答案是否一致。

参考:

- ▶ 第 N 步 $\approx 5K + 4K \times N$; $N = 30$ 时 $\approx 125K$, 窗口**基本装满**。
- ▶ 装满后按第二讲的滑动窗口, 最先被舍弃的是**最早的内容**——恰恰是开场那句“别动设定值”。

不做任何管理, 一个像样的排查任务, 半小时就能把 128K 撑爆。

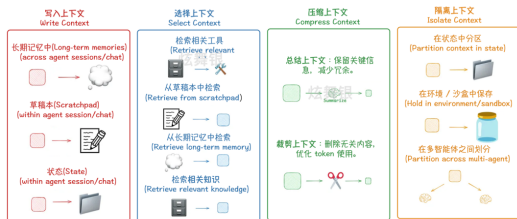
实际系统里系统提示通常不会被滑掉 (会被固定在最前面), 但历史里的关键约束会。

塞太满，不只是“忘”：四种常见的“上下文病”

名称	症状	值班助手里的样子
Poisoning 中毒	错误信息被写进上下文, 后面步步基于它推理	某次 DCS 工具返回了一个坏点 (传感器故障值), 之后的每一步都拿它当真
Distraction 分心	噪音淹没关键信息	20 万个数据点里, 那条“联锁值 = 25 °C/h”被淹没
Confusion 混淆	无关上下文干扰回答	上午排查二号反应器的记录还在窗口里, 它把两台的参数搞混
Clash 冲突	上下文内部自相矛盾	早期规程片段说“先降量”, 后来检索到的新版说“先切换”, 它不知听谁的

所以上下文工程要管的, 不只是**装得下**, 还有**装得对**、**装得干净**。

上下文工程：一句定义，四个动作



图源：LangChain, 色块都代表LLM Context

上下文工程 = 在**每一步**，往有限的窗口里放**恰到好处**的信息。

写入 关键信息记到窗口外
选择 只把相关的读回来
压缩 把要放的东西变小
隔离 不同任务互不干扰

配图：写入 → 选择 → 压缩 → 隔离 (来源：LangChain)

接下来三部分按这四步讲；第五部分讲窗口之外的**计算与成本**。

第二部分 · 写入上下文: 把关键信息记到窗口外面, 别指望窗口

人解决复杂问题时会**边想边记**: 中间结果写在纸上, 脑子只装当前这一步。

Agent 面对同样的问题:

- ▶ 窗口有限, 超过就遗忘;
- ▶ 有些任务需要多轮推理、反思, 甚至跨越好几次会话。

于是 Agent 系统引入两种“写下来”的机制:

Scratchpad 草稿本 短期 只为当前这个任务服务

Memory 长期记忆 长期 跨会话, 下次打开还在

“写入”不是让模型记住, 而是把信息放到一个**以后能找回来**的地方。

Scratchpad (草稿本): 当前任务内的笔记

目的: 执行任务时, 把中间思考、计划、计算结果暂存起来, **不让它们一直占着窗口。**

一个例子: 多智能体研究系统里, 主研究者先制定计划, 立刻把计划写进外部记忆——因为上下文超过 20 万 token 就会被截断, 计划必须在被挤出去之前“落盘”。

两种实现:

工具调用 把笔记写进外部文件、数据库或知识库; 需要时通过 RAG / MCP 读回。适合**长期、可共享**的任务笔记。

运行时状态 在当前会话里定义一个变量 (状态对象的字段), 临时保存计划或中间结果; 会话结束就没了。更**轻量**, 适合单次任务。

类比: 开会时手边那张草稿纸——帮你这一场不跑题, 散会就可以扔。

Long-term Memory (长期记忆): 跨会话还记得

目的: 让 Agent 记住你过去的习惯、任务历史和反思, 在**未来的会话**里继续使用。

两项代表性研究:

Reflexion 每次完成任务后做“自我反思”, 把成功或失败的经验写成一条记忆片段, 下次遇到类似任务先引用它。 [arXiv:2303.11366](#)

Generative Agents 定期把多个历史事件、反馈综合成“总结记忆”, 像人在一天结束时提炼当天的经历。 [arXiv:2304.03442](#)

已进入主流产品:

- ▶ ChatGPT 会自动生成长期记忆, 在多个对话之间记住你的偏好; 国内的豆包、Kimi 等也有类似的“记忆”功能;
- ▶ Cursor / Windsurf 等代码助手能记住你的项目结构与写作风格。

类比: 人脑的长期记忆区——几天后你仍记得上周定下的计划。功能细节随版本变化。

三层“记忆”放在一张图里



计算机类比: 内存 → 临时文件 → 硬盘

化工类比: 脑子里当前这一步 → 本班操作草稿 → 交接本 / 事故案例库

只有最上面的窗口是模型“真正看见”的; 下面两层都在窗口**外**, 靠**写**
入放进去、靠**选择**读回来。

越往上越贵、越快、越小; 越往下越便宜、越慢、越大——和计算机的内
存 / 硬盘一个道理。

化工同学早就在做的“写入”：值班交接班记录



一个班 8 小时, DCS 记录了上万条数据点

交接班时,你不会把 8 小时的曲线从头念一遍,只交接:

- ▶ 当前**关键参数** (温度、压力、液位是否正常);
- ▶ 本班**异常与处置** (何时报警、怎么处理的);
- ▶ **未完成事项**与下一班注意点。

这正是**写入**：把要跨“会话”（班次）的信息主动提炼后写进交接本；
曲线本身留在 DCS 历史库（长期存储），需要时再去**查**（下一部分的“选择”）。

Q:交接本上写“一切正常”和写“R-301 床层温升 $18\text{ }^{\circ}\text{C/h}$, 接近联锁值 25”, 哪个才算合格的“写入”?

小组讨论: 给实验室 AI 助手设计“该记什么”

课堂互动③

下面 8 条信息, 哪些该写进长期记忆, 哪些只留在本次任务的草稿本, 哪些根本不该? 4 人一组, 3 分钟, 分成三类并说出理由, 每组派人报一类。

- | | |
|----------------------------------|------------------|
| 1. 你习惯用 $^{\circ}\text{C}$ 而不是 K | 2. 今天这次实验的中间称量数据 |
| 3. 上次色谱柱老化的处理经验 | 4. 同组同学的手机号 |
| 5. 这次任务的分步计划 | 6. 导师要求报告必须附参考文献 |
| 7. 某次工具返回的报错原文 | 8. 你的校园网账号密码 |

参考分类:

- ▶ 长期记忆: 1、3、6 ——跨任务反复用得上的偏好、经验、规则;
- ▶ 草稿本: 2、5、7 ——只对当前任务有意义, 任务结束即可丢;
- ▶ 不记: 4、8 ——他人隐私与凭据, 不该进入任何 AI 的记忆。

判断标准就一句: 下次别的任务还会用到吗? 会 → 长期; 不会 → 草稿; 不该让它知道 → 不记。

短期草稿 vs. 长期记忆: 一张表

	Scratchpad 草稿本	Memory 长期记忆
活多久	当前任务 / 会话	跨会话, 天到月
存什么	计划、中间结果、待办	偏好、经验、反思、规则
存在哪	状态变量, 或临时文件	数据库、知识库、向量库
怎么读回	直接读状态, 或按名字取文件	检索 (相似度、时间、标签)
化工类比	本班的操作记录草稿	交接本、事故案例库、个人经验本

两者都不改模型参数 (那是第九讲微调的事), 改的只是**下一次往窗口里放什么**。

Q: 写是写下来了, 可一个任务写了几百条笔记、记忆库里有上千条经验——该读**哪几条**回来? $\Rightarrow$ 选择

第三部分 · 选择与压缩: 只取有用的, 再把它装小

上下文越积越多, 模型不可能每次“看完所有内容”。

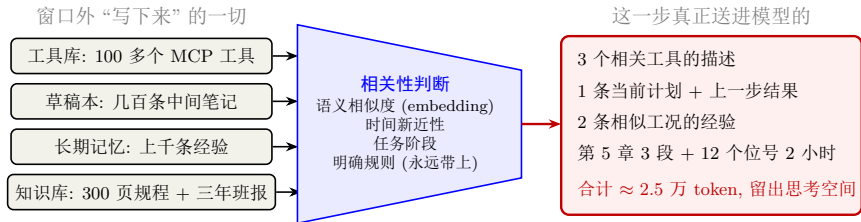
选择 = 在写下来的记忆、笔记、知识和工具里, 挑出与**当前任务**最相关的那部分。

Agent 在这一步常做四件事:

- 检索工具** 任务要联网、算数还是画图? 从工具库里挑合适的几个
- 读回草稿** 把上一步的计划、中间结果读回来, 不重复劳动
- 检索长期记忆** 调出过去的偏好、代码风格、总结性结论
- 检索知识** 从知识库 / 文档里找语义相近的片段——第四讲的 RAG

类比: 动笔写报告前, 先从笔记本、收藏夹和脑子里把“有用的资料”挑出来摆到桌上, 而不是把整个书柜搬来。

选择是一个漏斗：候选很多，进窗口的很少



漏斗的“滤网”每一步都重新算：任务阶段变了，读回来的东西也跟着变

左边是窗口外“写下来”的一切，右边是这一步真正送进模型的。
漏斗的滤网就是**相关性判断**：靠第二讲的向量相似度，也靠时间、任务阶段和明确规则。

“相关” 靠什么判断？四种信号

信号	怎么算	例子
语义相似度	问题和候选都转成 embedding, 比向量距离	“温升偏快” 召回规程里 “床层温升异常” 一节
时间新近性	越新的越可能相关	优先最近 2 小时的 DCS 数据, 而不是上个月的
任务阶段	规划阶段读计划, 执行阶段读工具结果	写结论时才把 “历史相似案例” 读回来
明确规则	有些内容永远带上, 不参与筛选	安全规则、“不得建议改设定值” 固定在系统提示里

工具也要选: 一个 MCP 服务器可能挂着上百个工具, 光是工具描述就能占几万 token。

做法是先按任务检索出相关的三五个, 再把它们的描述给模型 (回想第六讲)。

化工同学的版本: 选择性检索, 而不是搬整个历史库

问题还是那句: “三号反应器今晚温升为什么偏快?”

	全部塞进去	选择之后
DCS 数据	整套装置 800 个位号, 24 小时	该反应器相关 12 个位号 , 最近 2 小时
操作规程	300 页全文	第 5 章 “反应器温升异常” 3 段
历史工况	近三年全部班报	相似工况 2 个案例 的结论
合计	远超任何窗口	约 2.5 万 token, 留出思考空间

选择的依据是**工艺关系** (哪些位号与这台反应器有关) + **时间** + **语义** (哪一节规程在说这件事)。

Q:位号之间的工艺关系, 模型自己知道吗? 这类“领域结构”该由谁提供给它?

压缩上下文：行李箱就这么大，怎么多装点？

压缩 = 在不丢失关键信息的前提下，用更少的 token 装下更多内容。

方式	做法	常用于
Summarize 摘要压缩	让模型把一段内容提炼成要点，token 数大幅减少	长文档、会议记录、早期对话轮次
Trim 裁剪压缩	直接删掉无关或重复的部分，一个字不改	连续对话、多轮思考、工具返回的原始数据

很多产品和 Agent 框架的默认做法：对话超过某个阈值，就把**早期轮次自动摘要成一段**，只完整保留最近几轮。

摘要要是模型生成的，它也可能漏掉或改写关键数字——这个问题第五部分再谈。

压缩前后,窗口长这样



上排: 10 轮历史原样保留, 3 万 token; 下排: 前 7 轮摘要成 1 段, 工具返回只留结论, 最近 3 轮原样——不到 1 万。

省下的空间,才是模型**思考**和**装新信息**的地方。

化工同学早就在做的“压缩”：DCS 历史数据与趋势摘要

原始数据有多大：一个位号每秒采样一次，一天 86 400 个点；800 个位号一天近 7 000 万个点。

历史库怎么存 (裁剪)：死区压缩 / 旋转门算法，只保存“变化超过阈值”的点，平稳段几百个点只留两头。

班报怎么写 (摘要)：均值、峰值、越限次数与时刻、报警次数——一行字代替一天曲线。

	DCS	上下文工程
裁剪	死区压缩，删平稳点	删重复轮次、删原始返回
摘要	班报趋势摘要	早期对话摘要成一段
原件	历史库随时可查	原始记录写在窗口外

两边遵守同一条原则：**窗口 (班报) 里放摘要，原件留在外面，需要时再调。**

Q:查一次报警的真实原因时，你会只看班报，还是回历史库调秒级曲线？
对 AI 摘要该持同样的态度。

结构化上下文: 同样的信息, 摆整齐了模型更容易抓住

自由文本 (实验记录常见写法)	结构化模板
今天做了催化剂评价, 反应温度大概 380 度, 压力 2 MPa, 中间换了一次进料泵, 产物色谱下午 3 点出的, 转化率算出来 62%, 感觉比上次低, 可能是催化剂没活化好, 明天再做一组。	目的: 催化剂 C-12 活性评价 条件: 380 °C, 2.0 MPa, 空速同上次 异常: 10:40 更换进料泵 结果: 转化率 62% (上次 71%) 判断: 疑活化不足 (待验证) 待办: 重做一组, 活化 4 h

结构化的好处: **定位快** (要找“条件”就看那一行)、**省 token** (没有“大概”“感觉”)、
好裁剪 (整块删)、**好检索** (按字段查)。系统提示分节、工具返回用 JSON, 都是同一个道理。

动手改写 (两人一组, 3 分钟)

课堂互动④

把下面这段值班记录改写成结构化模板 (字段: 时间 / 位号 / 现象 / 处置 / 当前状态 / 待办), 然后数一数字数, 和原文比。

“大概十点多的时候 R-301 的床层温度涨得有点快, 看了一下每小时差不多涨了 18 度, 联锁是 25, 就先把进料量降了百分之十, 又观察了半个小时, 温升回到 10 度以下了, 不过原因还没查清楚, 怀疑是原料性质变了, 下一班记得让化验室补一个原料分析。”

参考答案:

时间: 22:10 ~ 22:40

位号: R-301 床层温度

现象: 温升 18 °C/h (联锁 25)

处置: 进料量 -10%

当前: 温升 < 10 °C/h, 原因未明

待办: 化验室补原料分析 (疑原料性质变化)

Q:原文哪些词被删掉了却一点信息没丢? 哪个数字是**绝对不能**在改写中丢掉或改动的?

第四部分 · 隔离上下文, 以及一个追问: 窗口够大不就行了?

Agent 同时处理多个任务、或者并行思考时, 要防止不同任务的上下文**互相干扰**。

状态分区 给每个任务单独开一个状态空间, 互不污染——像编程里的局部变量。

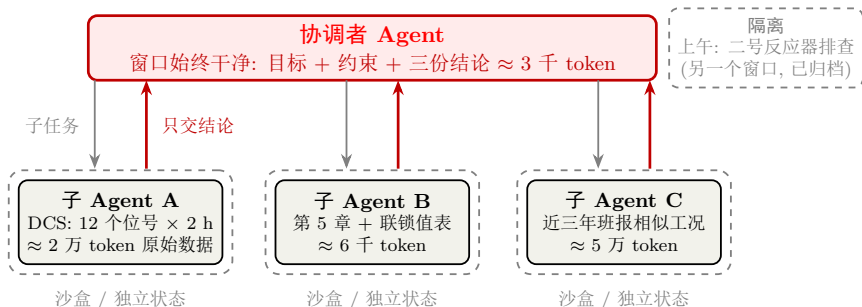
沙盒环境 外部函数、脚本在隔离环境里跑, 只把结果交回来; MCP、Function Call 都在安全沙盒中执行 (第五、六讲)。

多智能体划分 Planner、Coder、Reviewer 各自持有独立上下文, 最后由协调者汇总 (第七讲的多智能体)。

类比: 公司各部门有自己的文档、会议记录和数据库, 为同一个目标工作, 但互不翻乱对方的桌子。

化工版: 各车间各记各的操作台账, 调度室只收**日报**; 新控制逻辑先在**仿真系统**里验证, 不直接上 DCS。

隔离长什么样：每个子任务一个窗口，只把结论交上去



三个子 Agent 各自读了几万 token 的原始数据, 但交给协调者的只是**几百 token 的结论**;

协调者的窗口因此始终干净——上午排查二号反应器的记录, 不会混进三号的推理里。

追问：现在有 100 万 token 的模型了，把窗口开大不就完了？

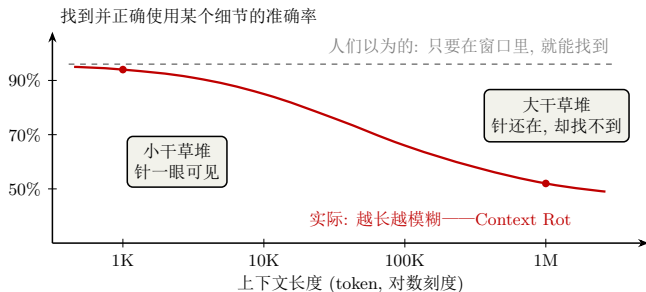
时期	典型上下文	大约相当于
2020 (GPT-3)	2K	一篇 3 页短文
2023	32K ~ 128K	一本 200 页的书
2024 ~ 2025	200K ~ 1M	一整套操作规程 + 一年班报

数字随版本更新变化很快，以各家文档为准。

窗口确实在变大，但有三个“但是”：

1. **贵**：每一轮都要重读全部输入，成本随长度线性增长（第五部分还有一个平方项）；
2. **慢**：首个 token 的等待时间随输入长度明显增加；
3. **效果不升反降**：塞得越多，模型对细节的把握反而越模糊——下一页。

Context Rot (上下文腐烂): 干草堆越大, 针越难找

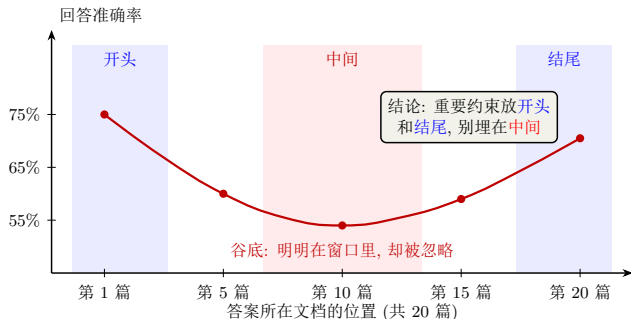


人在信息过载时会注意力分散; 模型面对极长上下文也会“认知漂移”: 失去焦点, 混淆不同片段的来源。

机制上, 注意力被**稀释**, 相关性权重分配变乱——信息明明“在窗口里”, 模型却聚焦不到、提取不出。

更大的窗口 $\neq$ 更强的理解力。“看得更多”和“看得清”是两件事。

Lost in the Middle (中间丢失): 位置也重要



实验: 把答案所在的文档放在一堆文档的不同位置, 再提问。关键信息在**开头**或**结尾**时准确率高, 放在**中间**时最低, 曲线呈 U 形 (arXiv:2307.03172)。

对我们的启示很具体: **重要约束放开头 (系统提示) 和结尾 (紧挨着问题), 别埋在中间。**

曲线为示意; 不同模型和版本的幅度不同, 但趋势被反复复现。

化工案例：整本规程贴进去，反而答错



300 页的操作规程，关键联锁值
在第 137 页的一张表里

做法 A：把 300 页《加氢装置操作规程》整本贴给模型（约 30 万 token），问：“反应器床层温升超过多少必须紧急泄压？”

结果：它给出一个相近但错误的数字——来自第 3 章另一台设备的表格。联锁表在全文正中间，正好落在 U 形曲线的谷底。

做法 B：只贴第 5 章“异常处理”+ 联锁值表（5 页，约 6 千 token），并在问题末尾重复“只以联锁表为准”。

结果：答对，并能指出表中的行号。

长上下文不能替代选择；而且无论 A 还是 B，联锁值这种数字最后都要回原文核对。

现场演示 + 预测：把一句话埋在中间，它还听吗？

课堂互动⑤

在 **Kimi** 或 **DeepSeek** 里贴一段约 3000 字的文章 (任选一篇你手边的长文)，在**正中间**插入一句“回答时请在结尾加上编号 CE-08”，然后问一个关于全文的问题。

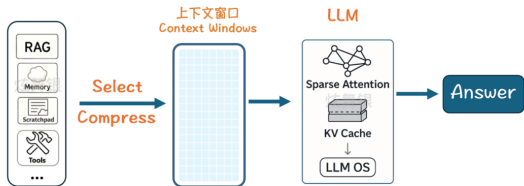
先举手预测：它会不会加编号？再实测。然后把同一句挪到**最后一行**，再问一次，对比。共 5 分钟。

记录三件事：

1. 埋在中间时，编号有没有出现？回答内容对不对？
2. 挪到结尾后呢？两次的差别说明了什么？
3. 如果把这句话换成“不得建议修改设定值”这类安全约束，你还敢把它埋在中间吗？

结果随模型和版本不同，有时两次都会加。重要的不是结论，而是学会**自己做这种测试**再决定怎么放信息。

四步做到位之后, 新的瓶颈才显现



配图: 选好、压好的信息进入窗口后, 还要靠计算层“算得动”

写入、选择、压缩、隔离的目标, 都是让**最有价值**的信息进窗口。

但窗口装满之后——文字、图片、代码、音频都挤在同一个空间里——问题从

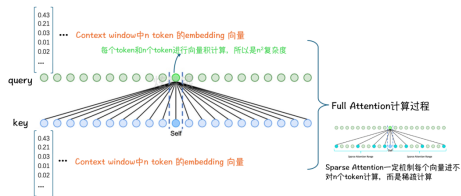
“怎么放进去”

变成了

“放进去以后, 模型怎么高效地算?”

这就是第五部分: 稀疏注意力、KV Cache, 以及它们和你的账单有什么关系。

第五部分 · 窗口之外: 计算层, 以及成本与效果的权衡



配图: 全注意力 (左) vs. 稀疏注意力 (右)

第一讲的注意力: 窗口里 N 个 token, 每一个都要和其他所有 token 比一次“我和你有没有关系”——约 $N \times N$ 次计算。

窗口从 8K 涨到 128K, 是 16 倍; 计算量涨的是 $16^2 = 256$ 倍。这就是上一部分说的“平方项”。

稀疏注意力 (Sparse Attention): 不再“全看”——只算局部窗口内的 token, 对全局只留少量“锚点”, 忽略大量无关部分。

计算量大幅下降; 更重要的是, 模型从“能看”变成“会看”: 在冗长信息里抓关键线索。

KV Cache (上下文缓存): 别把前面的对话从头再算一遍

模型逐个 token 往后生成。没有缓存时, 每生成一个新 token, 都要把它和之前所有 token 重新算一遍注意力——旧 token 之间的关系明明没变, 却被重复计算。像人每说一句话, 都把前面的对话从头想一遍。

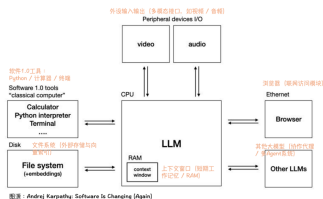
KV Cache: 把每一层算好的 Key、Value 缓存起来; 生成下一个 token 时直接复用, 只算新 token 对旧 token 的注意力, 不再重算整段。

- 快 推理速度提升数倍, 支持“流式”一个字一个字往外吐
- 省显存 不必反复加载旧 token
- 省钱 API 的“缓存命中”输入价格远低于未命中 (DeepSeek 文档: 上下文硬盘缓存)

对你的直接启示: 不变的内容 (系统提示、规程片段) 放在最前面, 每一轮都能命中缓存。

它不是长期记忆, 只是避免重复劳动的临时加速层; 具体价格随版本变化。

LLM OS: 把大模型当成一台“新型计算机”



配图: Andrej Karpathy 的 LLM OS 示意

计算机	大模型系统
CPU	LLM 本身
内存 (RAM)	上下文窗口
硬盘	文件系统 / 知识库 / 向量库 (长期记忆)
软件	计算器、Python、各种工具
网络	浏览器、其他 LLM
外设	音频、视频等多模态输入输出

Karpathy 的比喻: Prompt Engineering 是让模型理解你的一句话; Context Engineering 是让模型理解你所在的**世界**。

上下文工程的本质, 就是管理这台由语言驱动的计算机: **内存里放什么, 硬盘上存什么, 何时调用什么。**

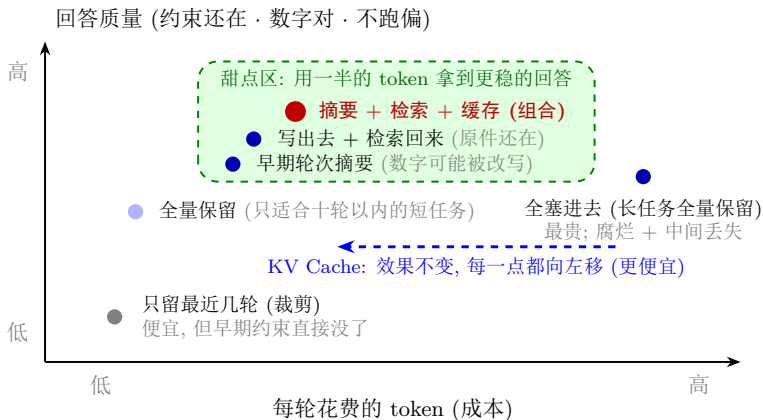
成本与效果的权衡：五种策略放在一张表里

策略	消耗	丢信息风险	难度	适合
全量保留	高, 递增	低 (直到被挤出)	无	十轮以内短任务
裁剪 (只留近几轮)	低	高 (早期约束没了)	低	闲聊, 无跨轮约束
早期轮次摘要	中	中 (数字可能走样)	中	多步 Agent
写出去 + 检索回来	低 ~ 中	低 (原件还在)	高	长期任务
KV Cache 缓存	不变	无	低	所有场景

没有一种策略在每一列都最好; 实际系统通常是**组合**:

固定内容前置命中缓存 + 早期轮次摘要 + 原始数据写出去按需检索 + 关键约束每轮重放在末尾。

画在坐标上：花多少钱，换多好的回答



压缩不等于可以不核对：用 AI 记忆的四条边界

- ▶ 让 AI 摘要的交接记录、文献要点，**关键数字**一律回原始数据 / 原文核对再用——摘要要是模型生成的，它会漏、会改；
- ▶ 他人的隐私、账号密码、未公开的实验数据，**不写进任何 AI 的长期记忆**，也不贴进对话；
- ▶ 用长上下文之前先算一笔账： $\text{token 数} \times \text{轮数} \times \text{单价}$ ，别用 100 万 token 的窗口干 5 千 token 的活；
- ▶ 报告、论文里引用 AI 整理的内容，必须能**追溯到原始出处**，否则不引用。

Q:讨论 (2 分钟): 互动③里那 8 条信息, 如果 AI 助手是全实验室共用的一个账号, 你的分类会变吗? 哪几条会从“长期记忆”挪出来?

下一页, 把今天的四步用到炼厂值班助手上。

案例分析: 为炼厂“AI 值班助手”设计上下文策略

课堂互动⑥

4 人一组, 5 分钟。回到开场的场景 (排查三号反应器温升, 30 步以上), 为它的上下文写四条规则, 每条一句话:

写入 什么 哪些信息要落到交接本 / 长期记忆, 而不是留在窗口里?

选择 什么 每一步只读回哪些位号、哪几节规程、哪几条历史?

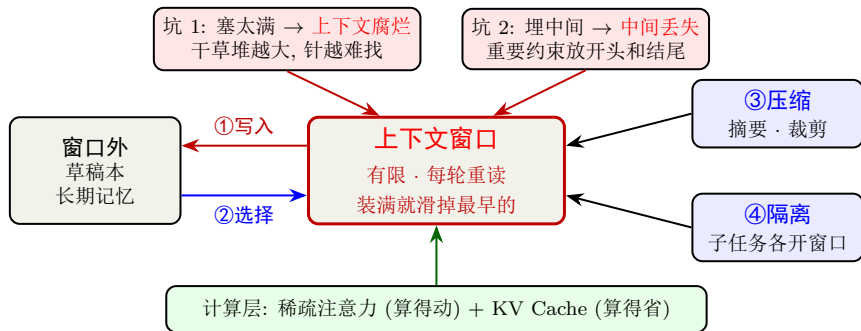
压缩 什么 历史轮次、DCS 原始返回各怎么压? 什么绝对不能压掉?

隔离 什么 二号、三号反应器的排查怎么互不串? 改设定值的动作放在哪里?

评价标准: 四步是否都落到了具体信息 (位号、章节、轮次) 而不是空话; “别动设定值” 这条约束在你们的方案里**放在哪**、每一轮还在不在。

选 2 组分享, 其他组只挑一件事: **哪一步最容易把关键约束弄丢?**

本讲总结: 一个窗口, 四个动作, 两个坑, 一层计算



LLM OS 的视角: 窗口是内存——内存里放什么, 硬盘里存什么

窗口里的东西每轮都被重读; 写入把关键信息放到窗口外, 选择把相关的读回来, 压缩把它变小, 隔离让任务互不干扰;
塞太满会腐烂、埋中间会丢失; 稀疏注意力和缓存让装进去的东西算得动、算得省。

随堂快测

课堂互动 · 3 道题

- Q1 (单选) 下列哪一项不属于模型一次推理的“上下文”?
- A. 系统提示 B. MCP 工具返回的数据 C. 模型的参数
D. RAG 检索到的片段
- Q2 (判断) 上下文窗口越大, 回答效果一定越好, 所以最好的上下文工程就是把窗口开到最大。
- Q3 (简答) 值班助手排查到第 30 步, 建议“下调设定值”, 违反了工程师开场的约束。用今天的词解释原因, 并给出两个改进办法。

答案: Q1 选 **C** (参数在推理时不变, 不属于上下文); Q2 **✗** 错误, 上下文腐烂、中间丢失、成本都随长度上升;

Q3 约束被滑出窗口或埋在中间被忽略; 改进: 约束放进系统提示并每轮重放在问题末尾 / 早期轮次摘要、工具返回只留结论以免撑爆窗口。

线上提交, 当堂出分。

课后任务与拓展

- 必做① 动手测: 复现互动⑤的“中间丢失”实验: 同一句指令分别放在开头、中间、结尾, 各测 3 次, 记录成表格, 写 150 字结论 (注明所用产品与日期)。
- 必做② 写方案: 把互动⑥的“值班助手上下文策略”整理成一页: 四步各一条规则, 并估算采用策略前后第 30 步的上下文 token 数。
- 选做③ 算账: 阅读 [DeepSeek 上下文缓存说明](#), 假设系统提示 5K token、对话 100 轮, 估算“固定内容前置”能省下多少输入费用 (写出计算过程)。

拓展阅读: [Lost in the Middle](#) (中间丢失的原始论文); [Reflexion](#) 与 [Generative Agents](#) (长期记忆的两项研究, 选读)。

下讲预告: Fine-Tuning —— 让 AI 更懂你的需求

Prompt、RAG、上下文工程
都用了, 还不够
微调改的是什么

全参数微调 vs. LoRA

数据集怎么准备

什么时候不该微调

什么样的需求非改模型本身不可

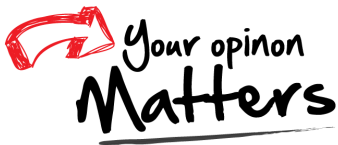
第二讲说过: 改参数, 不是改输入

为什么几张显卡也能微调一个几十亿
参数的模型

几百条高质量问答, 胜过几万条垃圾

知识会更新、需要出处的场景, 还是交
给 RAG 和上下文工程

Q: 课前思考: 你希望 AI “更懂” 你专业里的哪一件事? 这件事是靠喂资料就能解决, 还是得改它本身?



第八讲 · Context Engineering —— AI 如何记住重要信息
周天航 · 中国石油大学 (北京)

下课。请完成课后任务, 下次课带来你的“中间丢失”实验记录表。